

Plan: Generation 1 — an interpreter that matches the specification

spec/ALGOL-24.md is written and every one of its 260 rules is claimed by a case: a program in conformance/ , a refusal in refusals/ , or a reproduction in defects/ . The conformance pass decided what the language *should* do. This plan is the work of making the interpreter do it.

31 defects stand between the two. Each one is a rule the specification states and the interpreter does not implement, with a case in defects/ that passes while the fault persists and turns red the moment it stops.

0. The road to v1

		Ends when
Gen 0	The specification is written and reviewed.	The spec is read carefully and signed off. Merge to main , tag Gen 0 .
Gen 1	The spec is authoritative and conformance is enforcement. Work through the defects until there are none.	defects/ is empty. Tag Gen 1 .
Gen 2	Turn on the compiler. Annex C's divergences become defects and are worked through the same way.	Conformance passes under both processors and no defect remains. Tag Gen 2 .
v1	New language features and a runtime library, prioritized, each staying within conformance.	Release.

⚠ **What changes at the Gen 1 → Gen 2 boundary is the corpus doctrine itself.** Today a case is classified by one question — *is the interpreter right?* — and compiler gaps are reported but do not fail the run (conformance/README.md). At Gen 2 that relaxation ends: ./conform.sh --strict becomes the gate, and each Annex C entry earns a defect of its own. The classification question does not change, because by then the interpreter matches the specification and "is the interpreter right?" is answered yes everywhere; what changes is that the compiler is no longer excused.

⚠ **Four Annex C entries will not survive to Gen 2**, and should not be written up as compiler defects when the time comes:

- **C-1** closes with DEF-24 — giving the root a module identity.
- **C-12** closes with DEF-17 — the language stops allowing what the compiler already refused.

- **C-4** and **C-9** *reverse*: [SRC-011] and [RT-017] make the compiler right and the interpreter the one to change, so they are Gen 1 work (DEF-02, DEF-26) rather than Gen 2 work.

⚠ **Nothing after Gen 0 may change a rule without a reason recorded in the specification.** The point of signing off is that the document stops being a description of the implementation and starts being the thing the implementation is measured against — so a defect is closed by changing the code, and a rule is changed only by deciding to change the language.

1. What "done" means

```
./conform.sh --interpreted    every case green, and defects/ empty
./spec/spec.sh               0 rules NOT YET IMPLEMENTED, 0 PARTLY
./test.sh                    the compiler's own suite still passes
./fixedpoint.sh              the seed is in sync and the fixed point holds
```

⚠ `fixedpoint.sh` **is the constraint, not a formality.** Every change here is a change to a compiler written in the language it compiles, and it must be reseeded through the old binary before the new behaviour can be relied on. A change to how the compiler emits its own code takes **two** generations to settle — generation 1 holds the new rules written by the old emitter — which is why the script iterates and why one reseed is not always enough.

⚠ **The compiler must keep compiling.** Compiler gaps going red is expected and fine; `algc` failing to build itself is not, because it is what produces the generation that closes them.

2. Order, and why it is not free

Most defects are independent. Seven are not, and the dependencies were found while writing the cases rather than by reading the code.

DEF-12 before DEF-09. [VAR-006] routes every untyped-to-typed crossing through `as`. Tightening the assignment path while `as` still checks nothing moves the hole rather than closing it.

DEF-10 with DEF-19. A parameter is an assignment context, so it must **widen** as well as match. Enforcing parameter types before widening works would start refusing a Char argument reaching a String parameter, which is correct code.

DEF-27 with DEF-10. Once widening works, `Max` promoting is the rule the operators already use rather than a special case in one built-in.

DEF-02 needs `Console.a24` **renamed first.** It declares a module-level `const INFO` and an object member procedure `Info`, and likewise `ERROR` and `Error`. Under folded lookup the member shadows the constant inside `Info` itself and the compiler breaks.

DEF-04 needs SAMPLE rewritten first. `compiler/Main.a24` 's built-in sample program — what `alg` runs when given no arguments — is written with `print`, so removing the statement breaks it.

DEF-14 is the hash, not the comparison. A Map and a Set bucket by a hash, so an Integer and a Double of one value must land in the same bucket or `Contains` answers false for a key the Map holds. Changing only the comparison leaves the bug.

DEF-06 splits. The range moves with DEF-01, since a Char wider than a byte and a String of characters are the same change. The *diagnostic* half — a scan error escaping as an uncaught raise with no line — is independent and worth doing whatever the range turns out to be.

⚠ **DEF-15 must not be fixed by copying the compiler.** The compiled back end hoists every top-level name; [DCL-006] wants functions and classes hoisted and [DCL-016] wants variables not. One mechanism, correct for one kind of declaration and not the other.

3. Suggested sequence

Wave 1 — local, no dependencies, each a few lines. DEF-07 (unterminated string reports the opening line), DEF-13 (unknown type name in `is`), DEF-18 (a procedure may not `Exit` a value), DEF-20 (name the class in an inheritance error), DEF-21 (publish `Ordinal`), DEF-25 (`Length` refuses a collection), DEF-26 (a String answers `.Length`), DEF-28 (a file that cannot be read exits non-zero), DEF-31 (a one-character test name), DEF-06's diagnostic half.

⚠ **DEF-29 belongs in this wave and is worth doing first.** Every subsequent wave produces type errors while it is being debugged, and today each one says `Type mismatch!` and nothing else. Fixing the diagnostic first makes the rest of this plan cheaper to execute.

Wave 2 — the type system, in order. DEF-12, then DEF-16 (`as` binds tightly — a latent oddity becomes a live fault once the cast checks), then DEF-10 with DEF-19 and DEF-27, then DEF-09.

Wave 3 — parsing and scoping. DEF-03 (identifier marks), DEF-17 (declaration as an unbraced body — ⚠ this one *removes* C-12), DEF-15 (hoisting), DEF-11 (`of` on every collection), DEF-05 (overflow: the literal half is small and local, the arithmetic half touches every operator).

Wave 4 — names and modules. DEF-22 and DEF-23 are the same shape — accept the declaration, refuse the ambiguous bare use — and should be done together. DEF-24 (root cycle) ⚠ also removes C-1, the only known case of a valid program having no compiled form. DEF-02 (case-insensitive names) after `Console.a24` is renamed; ⚠ it reverses C-4, making the compiler right and the interpreter the one to change.

Wave 5 — the large ones. DEF-01 (text is characters, not bytes) is the biggest change the specification asks for and touches the scanner, both runtimes, `Length` / `Copy` / `Pos` / subscript, `Ord` / `Char` and the emitter's mangling. DEF-08 and DEF-06's range ride with it. DEF-14 (membership follows equality) is independent but needs the hash.

Wave 6 — the report. DEF-30 changes text both processors must reproduce byte for byte, so `compiler/Interpreter.a24` and `bootstrap/algol.c` move together. DEF-04 (remove `print`) after `SAMPLE` is rewritten.

4. Working rules

Fix the rule the case names, and no more. Each defect cites the rules it violates; the case in `defects/` is the acceptance test. When it stops reproducing, `conform.sh` reports `CHANGED` — that is the signal to promote it to `conformance/` and delete the Annex F entry, not a failure.

Reseed before believing anything. `bootstrap/*.c` is a fixed point. A change to `compiler/*.a24` that has not been reseeded is not yet real, and running `algolc` under `algolc` is unreliable for anything large.

A rule that names a field, a flag or a call order is describing the implementation. Two rules were written that way and had to be rewritten ([ERR-004] mandated a flag, [TST-014] described the compiler). If a fix makes a rule easier to state, the rule was probably describing machinery.

⚠ **Before recording any new case, ask what it would print if the implementation were already right.** If that is what you are about to record, the case proves nothing. See `conformance/README.md` , which carries this and the rest of the corpus doctrine.

5. Not in this generation

The compiler. 28 gaps in Annex C, expected to stay red throughout. They are generation 2, and `./conform.sh --strict` is the gate that will open it. Four will close as a side effect of this generation — C-1 with DEF-24, C-12 with DEF-17, and C-4 and C-9 reverse, becoming interpreter defects that this generation fixes.

Six planned generations in Annex H: alternate bases and digit separators, exponent notation, element types checked on insertion, a subscript operator, an iteration protocol, a computed property, ordering for Strings, operator overloading, and the collections as a unit written in Algol-24. ⚠ H-4, H-5 and H-6 are prerequisites for H-9, not companions to it.

Two decisions deliberately left open, recorded in Annex D rather than lost:

- Whether the built-in `Boolean` should become `type Boolean = (False, True) . Tabled`. ⚠ [ENU-011] collides with it — members bind bare, so any program declaring `True` , `False` , `Yes` or `No` as a member name would make that name ambiguous at every bare use — and `VAL_BOOL` is a distinct runtime tag where an enum member is an interned object.
- Whether `Integer 0` should be truthy, which would reduce truthiness to "only `nil` and `False` are false". It changes what existing programs mean, so it belongs to neither annex until someone takes it.

6. Still unbuilt

Screen output in test blocks. One of the three testing forms asked for at the start, and the only one never built: a buffer cleared before each test body and readable from it, so a test can assert on what a program printed. Nothing in this plan depends on it, and it is the natural first feature to add once the interpreter conforms.

⚠ The design question recorded at the time is still open: whether the buffer is a `Buffer` reachable from Algol-24 or an opaque native. A `Buffer` brings `Free` and its poisoning rule into the test runner's lifetime, which is a larger surface than the feature needs.

Continuous integration. There is none. `spec.sh`, `conform.sh`, `test.sh` and `fixedpoint.sh` all exit non-zero on failure and are ready to be gates; nothing runs them but a person. ⚠ `conform.sh --record` refuses to run when `CI` is set, which is the only preparation made so far.

7. What the conformance pass produced

For the record, and because the numbers say where the effort went.

Rules	260, none awaiting a case
Case files	169 — 108 conformance, 29 refusals, 32 defects
Rules citing a conformance program	196
Rules citing a refusal	30
Rules citing a defect	45
Probes	134

⚠ **Those three do not sum to 260, and are not meant to.** A case usually pins several rules — 1.7 on average, and `conformance/0058-exceptions.a24` covers five — and a rule that is *partly* implemented cites both a conformance program for the half that works and a defect for the half that does not. The number that must reach 260 is **accounted for**, which `spec.sh` reports separately and gates on. | Annex C — compiler divergences | 15 | | Annex D — advisory notes | 18, every one resolved | | Annex H — planned generations | 9 | | Checked tables | keywords, built-ins, grammar productions, the collection member matrix |

⚠ **Three rules were overturned by running them**, having been written from reading the code: [CLS-014] said the superclass check happens at construction (it happens at the declaration), [MOD-012] said circular imports do not work (they work), and D-15 invented a coincidence between `Length(L)` and `L.Length` that never occurs. All three erred the same way — more confident than the evidence supported.